

Sprawozdanie z projektu: TechStore – System zarządzania produktami

Spis treści

1. Opis projektu	2
1.1 Cel projektu.....	2
2. Funkcjonalności aplikacji.....	2
2.1 Wyświetlanie listy produktów	2
2.2 Dodawanie, edycja i usuwanie produktów	2
2.3 Promocje i inflacja cen	3
2.4 Efekty wizualne (confetti)	3
3. Środowisko uruchomieniowe	3
3.1 Wymagania systemowe i narzędzia.....	3
3.2 Konfiguracja bazy danych	5
3.3 Uruchomienie backendu (Express, Flask)	6
3.4 Uruchomienie frontendu (Angular).....	6
5. Interfejs użytkownika	6
5.1 Zrzuty ekranu aplikacji.....	6
5.2 Opis zastosowanego motywu.....	7
5.3 Responsywność (Bootstrap)	7
6. Struktura projektu	8
6.1 Opis struktury katalogów frontendu (Angular).....	8
6.2 Opis struktury backendu (Express.js i Flask).....	8
7. API aplikacji	9
7.1 Zastosowanie endpointów w aplikacji	9
8. Materiały dodatkowe	9
8.1 Link do repozytorium.....	9

1. Opis projektu

1.1 Cel projektu

Głównym celem projektu było poznanie i praktyczne zastosowanie nowoczesnych technologii webowych wykorzystywanych w budowie aplikacji typu full-stack. Po stronie backendu projekt obejmował naukę dwóch różnych technologii - Express.js opartego na Node.js oraz Flaska w Pythonie, natomiast po stronie frontendu projekt obejmował zapoznanie się ze frameworkiem - Angular

2. Funkcjonalności aplikacji

2.1 Wyświetlanie listy produktów

Aplikacja pobiera wszystkie produkty z bazy danych za pomocą metody pobierzProdukty() i wyświetla je w tabeli z kolumnami: ID, Nazwa, Producent, Kategoria oraz Cena. Po kliknięciu na wiersz tabeli produkt zostaje zaznaczony, a jego dane tadeją się automatycznie do formularza edycji.

Lista Produktów				
ID	Nazwa	Producent	Kategoria	Cena
1	RTX 4080 Supe	NVIDIA	Karty Graficzne	430.41 PLN
2	Core i9-14900	Intel	Procesory	236.79 PLN
3	Trident Z5 32Mb	G.Skill	Pamięci RAM	82309.86 PLN
4	Fury Renegade 2TB	Kingston	Dyski SSD	67.17 PLN

2.2 Dodawanie, edycja i usuwanie produktów

Formularz umożliwia wprowadzenie nazwy produktu, producenta, kategorii (wybór z listy rozwijanej) oraz ceny. Przycisk "Dodaj" tworzy nowy produkt w bazie, "Aktualizuj" zapisuje zmiany w zaznaczonym produkcie, a "Usuń" trwale usuwa wybrany produkt. Każda operacja zawiera walidację pól formularza.

Edycja / Dodawanie Produktu	
Wybierz produkt:	
2 - Core i9-14900	
Nazwa produktu	Producent
Core i9-14900	Intel
Kategoria	Cena (PLN)
Procesory	292.33
Dodaj Nowy	Aktualizuj
Usuń	

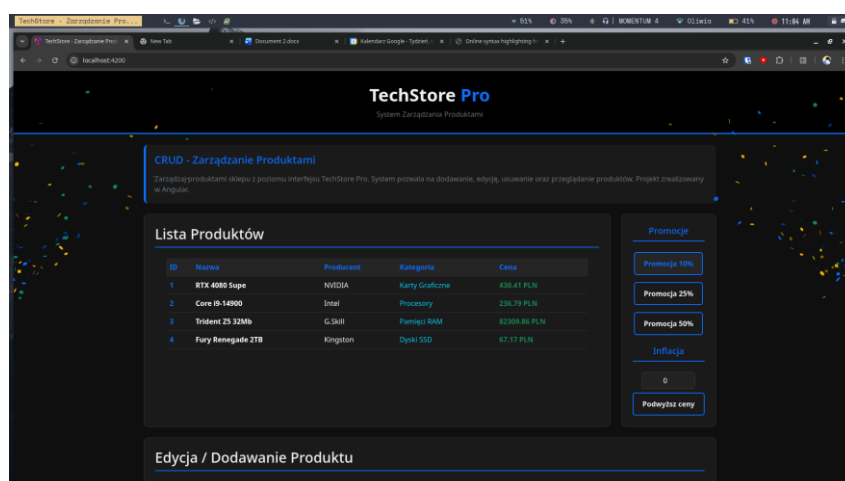
2.3 Promocje i inflacja cen

Sekcja promocji zawiera trzy przyciski zniżek (10%, 20%, 30%), które obniżają ceny wszystkich produktów o wybrany procent. Pole "Inflacja" pozwala na wprowadzenie własnej wartości podwyżki (od 2% do 100%) i zastosowanie jej do wszystkich produktów jednocześnie.



2.4 Efekty wizualne (confetti)

Po zastosowaniu promocji wyświetla się animacja kolorowego confetti wystrzeliwanego z obu stron ekranu przez 1 sekundę. Przy inflacji z góry ekranu opadają emoji 💎 i 🎉 przez 1.5 sekundy



3. Środowisko uruchomieniowe

3.1 Wymagania systemowe i narzędzia

Wymaganie	Minimalna wersja
System operacyjny	Windows 10 / Linux / macOS
Node.js	18.0 lub nowszy
Python	3.10 lub nowszy
MySQL	5.7 lub nowszy
Przeglądarka	Chrome, Firefox, Edge (aktualna wersja)

Wymagane biblioteki:

-> frontend:

Biblioteka	Wersja	Opis
@angular/core	21.x	Framework Angular
@angular/forms	21.x	Obsługa formularzy
axios	1.x	Komunikacja HTTP
@tsparticles/confetti	-	Efekty wizualne
bootstrap	5.x	Style CSS

-> Backend Express

Biblioteka	Opis
express	Framework webowy
cors	Obsługa CORS
mysql2	Sterownik MySQL

-> Backend Flask

Biblioteka	Opis
flask	Framework webowy
flask-cors	Obsługa CORS
pymysql	Sterownik MySQL

3.2 Konfiguracja bazy danych

Projekt wykorzystuje bazę danych MySQL o nazwie Hardware_OAnkiewicz. Poniżej podaje skrypt SQL tworzący bazę, tabelę oraz dodający dane początkowe.

```
CREATE DATABASE Hardware_OAnkiewicz;  
ALTER DATABASE Hardware_OAnkiewicz CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
USE Hardware_OAnkiewicz;
```

```
CREATE TABLE products (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  producent VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  price DECIMAL(10, 2) NOT NULL  
);
```

Struktura tabeli products:

Kolumna	Typ	Opis
id	INT AUTO_INCREMENT	Unikalny identyfikator produktu (klucz główny)
name	VARCHAR(100)	Nazwa produktu
producent	VARCHAR(100)	Nazwa producenta
category	VARCHAR(50)	Kategoria produktu
price	DECIMAL(10,2)	Cena produktu (do 2 miejsc po przecinku)

Dane początkowe

```
INSERT INTO products (name, producent, category, price) VALUES  
( 'RTX 4080 Super', 'NVIDIA', 'Karty Graficzne', 4999.00),  
( 'Core i9-14900K', 'Intel', 'Procesory', 2750.00),  
( 'Trident Z5 32GB', 'G.Skill', 'Pamięci RAM', 620.00),  
( 'Fury Renegade 2TB', 'Kingston', 'Dyski SSD', 780.00);
```

Konfiguracja połączenia w aplikacji

Express.js (db/baza_conn.js)	Flask (app.py)
<pre>const mysql = require('mysql2'); const link = mysql.createConnection({ host: 'localhost', user: 'root', password: '', database: 'Hardware_OAnkiewicz' });</pre>	<pre>app.config['MYSQL_HOST'] = 'localhost' app.config['MYSQL_USER'] = 'root' app.config['MYSQL_PASSWORD'] = '' app.config['MYSQL_DB'] = 'Hardware_OAnkiewicz'</pre>

3.3 Uruchomienie backendu (Express, Flask)

Express:

```
cd server_express
npm install
npm start
# Serwer działa na http://localhost:3005
```

Flask (bez wirtualnego środowiska):

```
cd server_flask
pip install flask flask-cors pymysql
python app.py
# Serwer działa na http://localhost:3005
```

Flask (wirtualne środowisko):

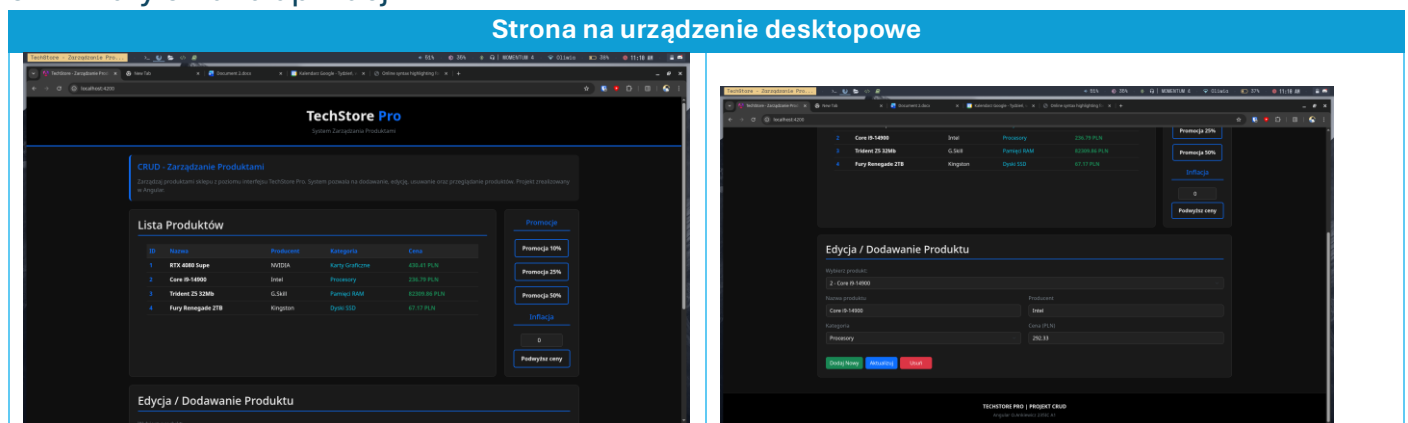
```
cd server_flask
python3 -m venv venv_oAnkiewicz_flask
.\venv_oAnkiewicz_flask\Scripts\activate
pip install flask flask-cors pymysql
python app.py
```

3.4 Uruchomienie frontendu (Angular)

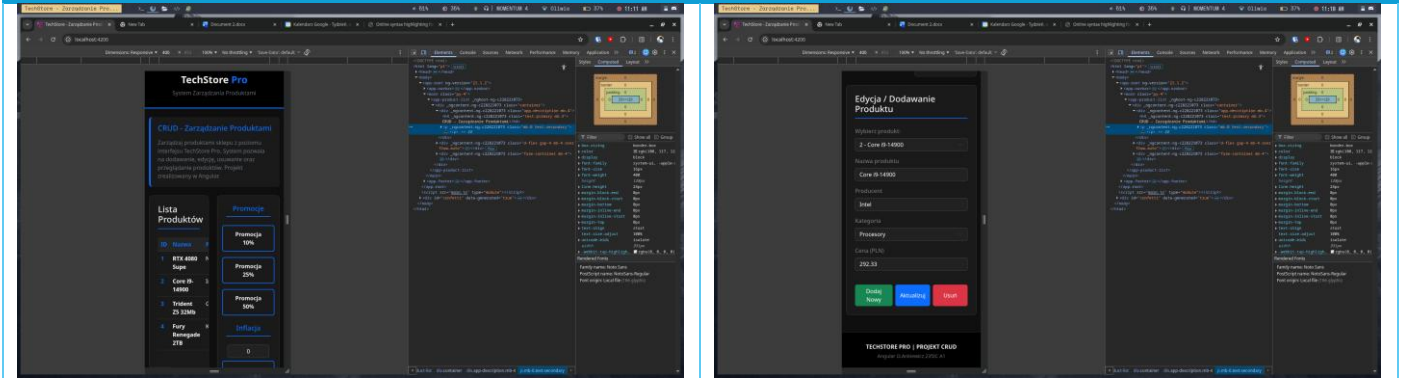
```
cd frontEnd_angular
npm install
npm start
# Aplikacja działa na http://localhost:4200
```

5. Interfejs użytkownika

5.1 Zrzuty ekranu aplikacji



Strona na urządzenie mobilne



5.2 Opis zastosowanego motywu

Aplikacja wykorzystuje ciemny motyw kolorystyczny (dark theme) oparty na odcieniach czerni i szarości. Tło główne ma kolor #121212, a elementy takie jak tabela, formularze i sekcja promocji używają ciemniejszego odcienia #1a1a1a z subtelną ramką #333. Tekst wyświetlany jest w jasnym kolorze #f0f0f0.

5.3 Responsywność (Bootstrap)

Aplikacja wykorzystuje system siatki Bootstrap 5 do zapewnienia responsywności na różnych urządzeniach. Główny kontener używa klasy container-fluid zapewniającej pełną szerokość ekranu, a układ strony oparty jest na klasach row i col dostosowujących się do rozmiaru okna przeglądarki.

6. Struktura projektu

6.1 Opis struktury katalogów frontendu (Angular)

```
frontEnd_angular/  
├── src/  
│   ├── index.html  
│   ├── main.ts  
│   ├── styles.css  
│   └── app/  
│       ├── app.ts  
│       ├── app.html  
│       ├── app.config.ts  
│       ├── app.routes.ts  
│       └── components/  
│           ├── navbar/  
│           │   ├── navbar.ts  
│           │   └── navbar.html  
│           ├── footer/  
│           │   ├── footer.ts  
│           │   └── footer.html  
│           └── product-list/  
│               ├── product-list.ts  
│               ├── product-list.html  
│               └── product-list.css  
├── angular.json  
├── package.json  
└── tsconfig.json
```

6.2 Opis struktury backendu (Express.js i Flask)

Express.js:

```
server_express/  
├── index.js  
├── package.json  
├── api/  
│   ├── router.js  
│   └── akcje.js  
└── db/  
    └── baza_conn.js
```


Flask:

```
server_flask/  
└─ app.py
```

7. API aplikacji

7.1 Zastosowanie endpointów w aplikacji

Metoda	Endpoint	Opis
GET	/products/	Pobiera wszystkie produkty
GET	/products/:id	Pobiera produkt po ID
POST	/products/	Dodaje nowy produkt
PUT	/products/:id	Aktualizuje produkt
DELETE	/products/:id	Usuwa produkt o podanym id
PUT	/products/zmianaCeny/:wartosc	Zmiana ceny (promocja/inflacja)

8. Materiały dodatkowe

8.1 Link do repozytorium

https://github.com/livcia/BWO_zaliczenie_przedmiotu